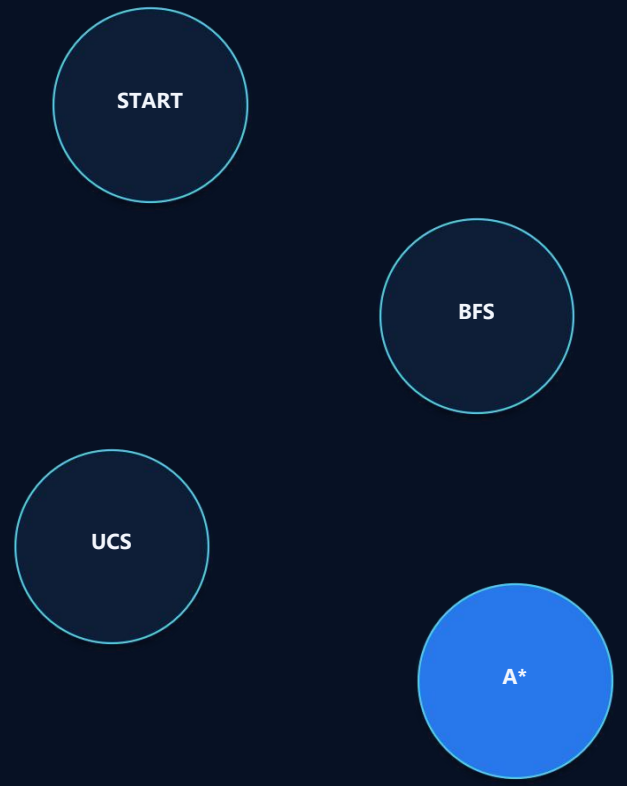
LECTURE 2

Uninformed Search, A* Search & Heuristics



BFS, DFS, UCS, A*

From search trees and fringes to a working Python A* navigator

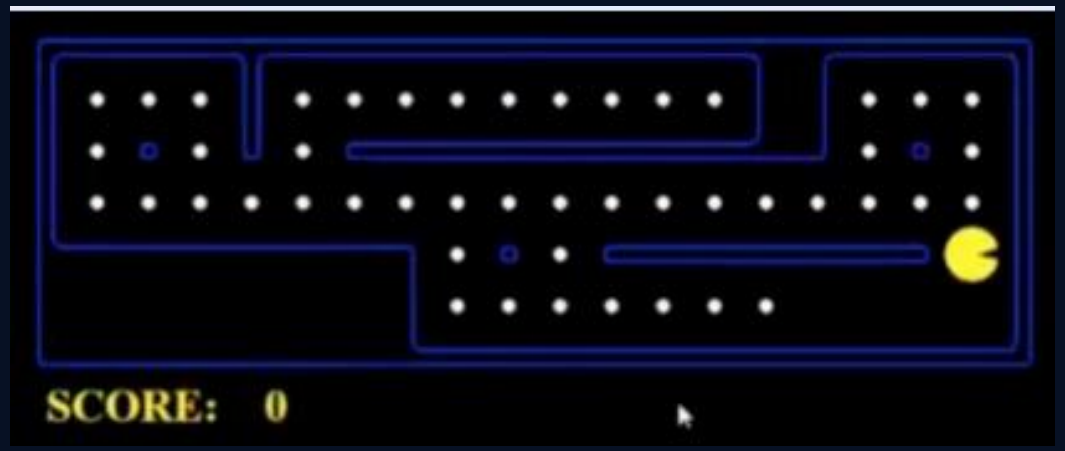
Learning outcomes

What students should be able to do

- Model states, actions, successors, goals, and costs
- Implement BFS, DFS, UCS, and A*
- Read Python operators used in search code
- Explain g(n), h(n), and f(n)
- Design and test heuristics
- Measure search behavior

What's in a State Space?

The world state includes every last detail of the environment



A search state keeps only the details needed for planning (abstraction)

Problem: Pathing
States: (x,y) location
Actions: NSEW
Successor: update location only
Goal test: is (x,y)=END

Planning agents

A planning agent asks: “What if I take this action?”

It uses a model of how the world changes, a goal test, and search over possible plans.

Examples: route planning, robot motion, games, puzzles.

Search problem

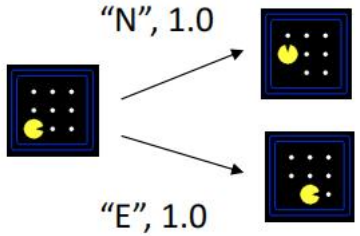
Core model

A search problem contains:

1. State space



2. Successor function: actions + costs



3. Start state

4. Goal test

A solution is a sequence of actions that transforms START into GOAL.

Why do we care about search?

The algorithms we'll discuss are incredibly widely applied

- Google Maps → find a route
- Pac-Man → find a path to food
- Robot navigation → reach a destination
- Puzzle solving → reach the target configuration
- Game AI → find a good sequence of moves
- Scheduling → find a valid schedule
- Network routing → find a path through computers

For example...

State space: Cities

Successor function:
Roads: Go to adjacent city with cost = distance

Start state: Astana

Goal test: Is state == Almaty?

Solution?



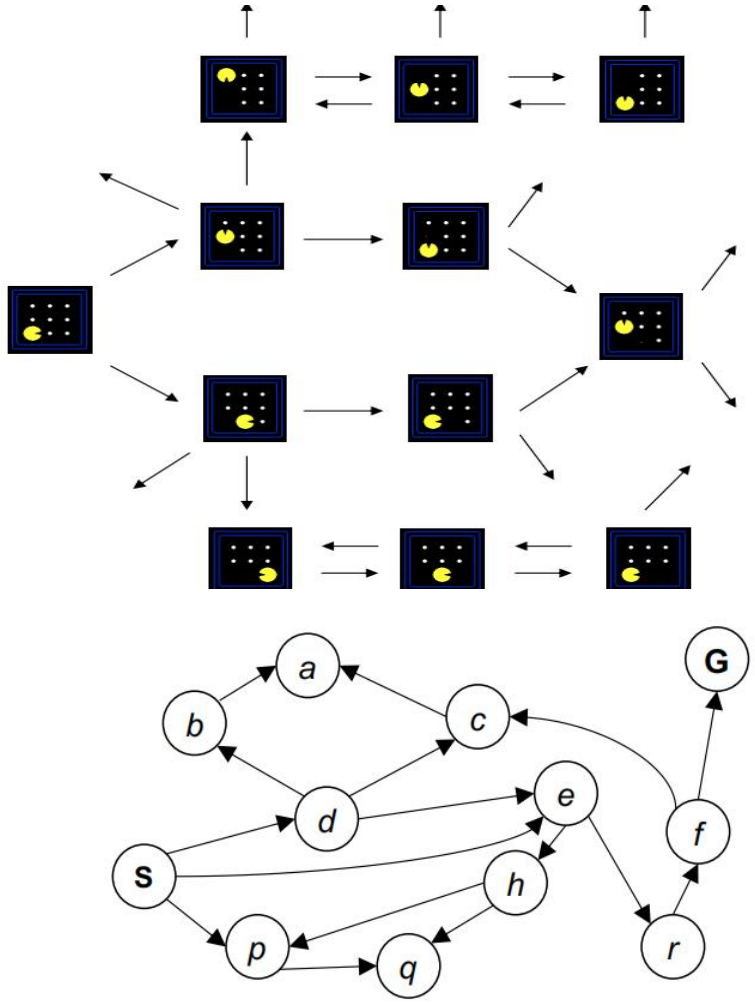
State-space graph

State space graph: A mathematical representation of a search problem

- Nodes are (abstracted) world configurations
- Arcs represent successors (action results)
- The goal test is a set of goal nodes (maybe only one)

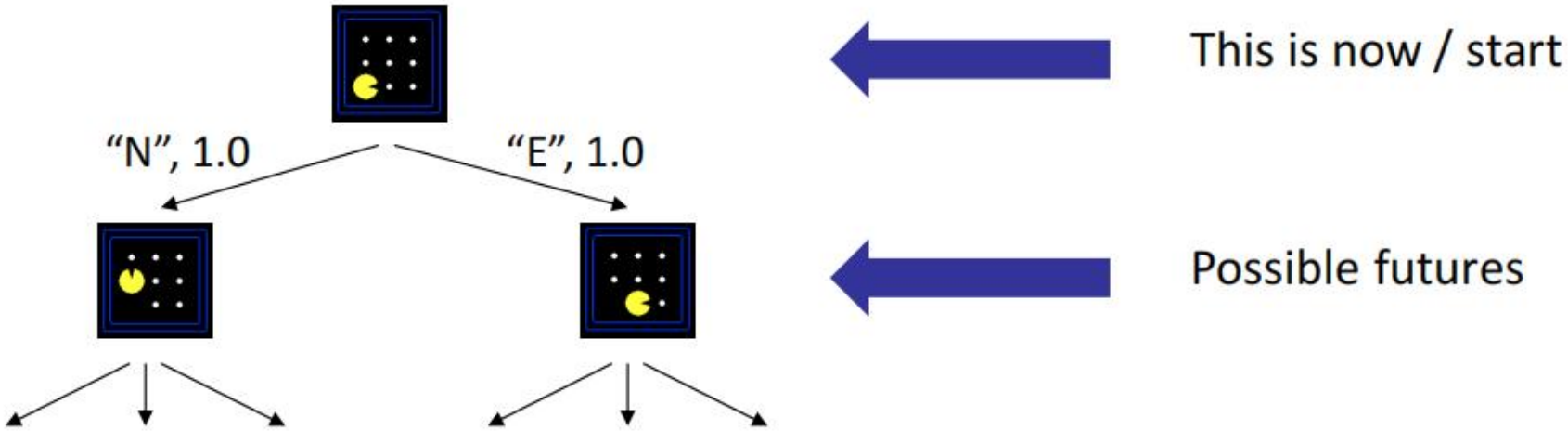
In a state space graph, each state occurs only once!

We can rarely build this full graph in memory (it's too big), but it's a useful idea



Tiny state space graph for a tiny search problem

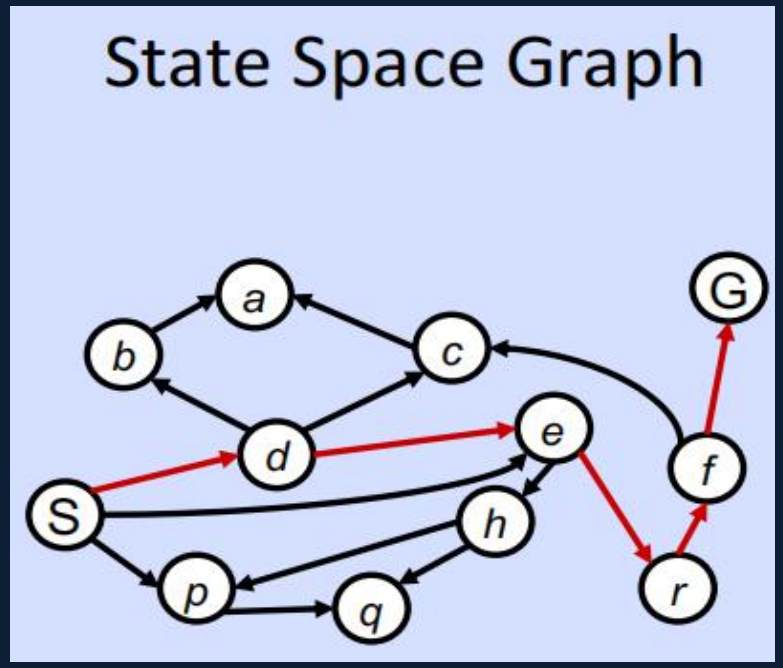
Search Trees



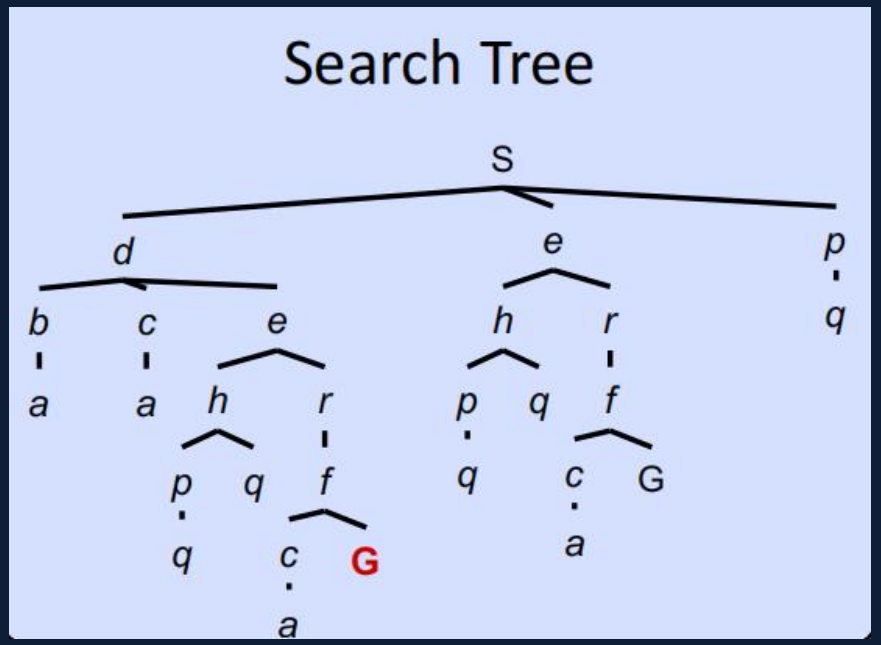
A search tree:

- **A "what if" tree of plans and their outcomes**
- **The start state is the root node**
- **Children correspond to successors**
- **Nodes show states, but correspond to PLANS that achieve those states**
- **For most problems, we can never actually build the whole tree**

State Space Graphs vs. Search Trees

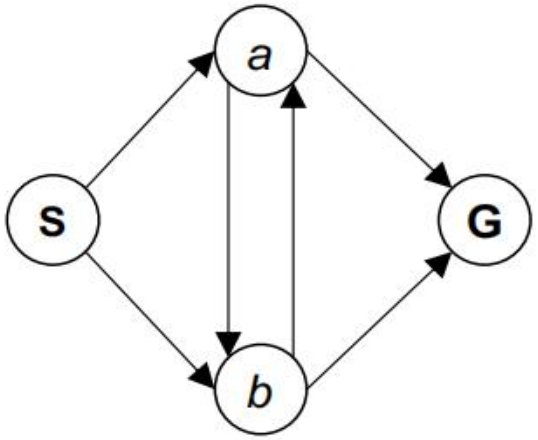


Each NODE in in the search tree is an entire PATH in the state space graph.

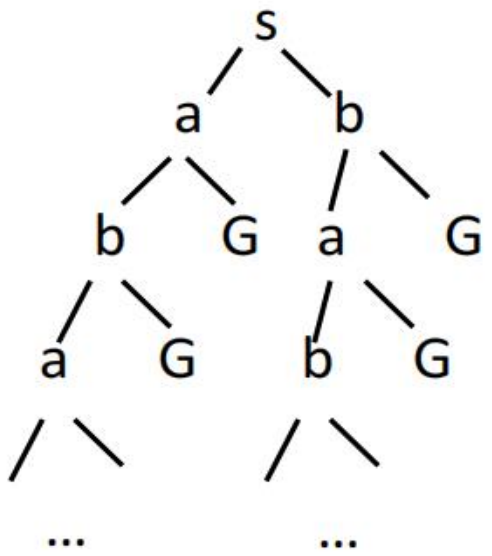


State Space Graphs vs. Search Trees

Consider this 4-state graph:



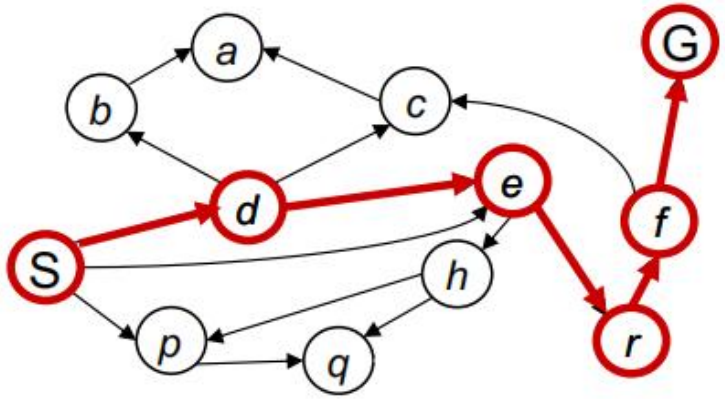
How big is its search tree (from S)?

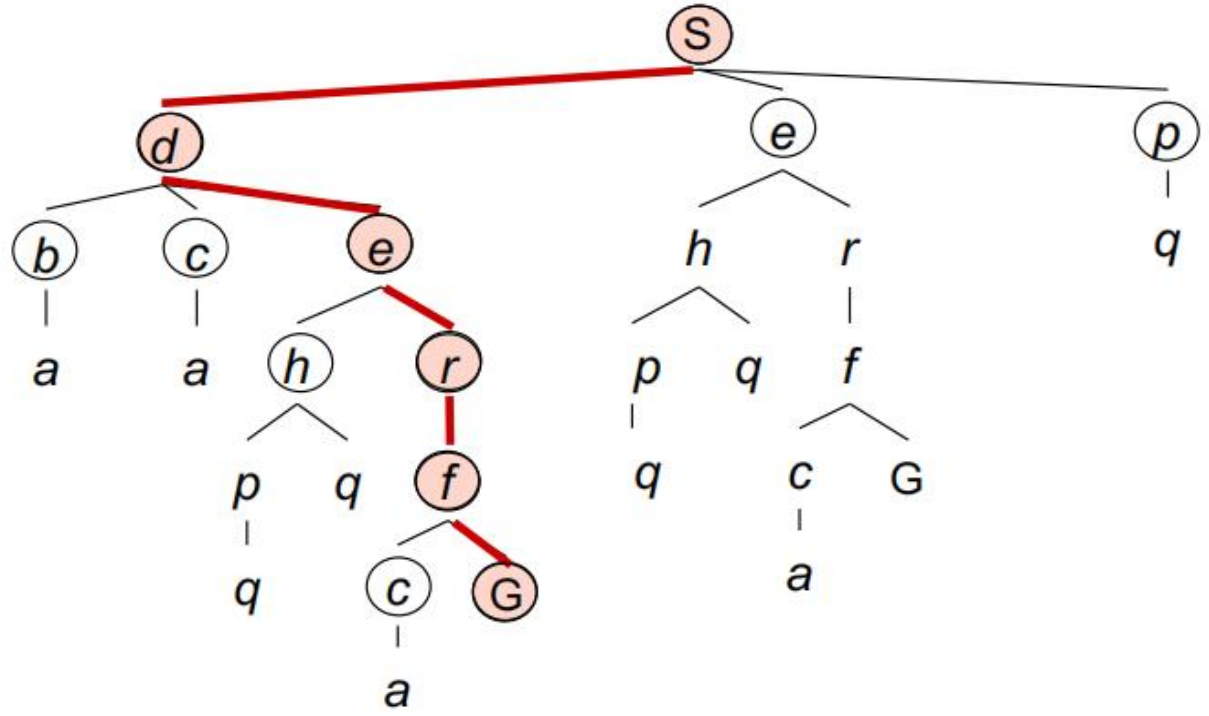




Important: Lots of repeated structure in the search tree!

Search Trees





~~S~~

~~S → d~~

S → e

S → p

S → d → b

S → d → c

~~S → d → e~~

S → d → e → h

~~S → d → e → r~~

~~S → d → e → r → f~~

S → d → e → r → f → c

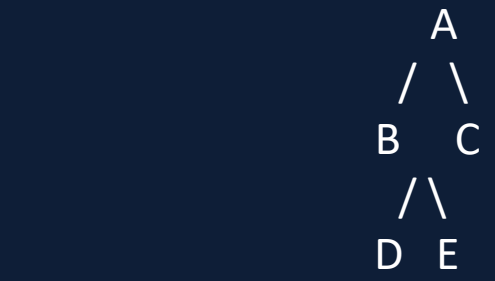
~~S → d → e → r → f → G~~

The frontier / fringe

The frontier contains partial plans waiting to be explored.

Main question: WHICH frontier node next?

DFS → deepest
BFS → shallowest
UCS → lowest g(n)
A* → lowest g(n)+h(n)



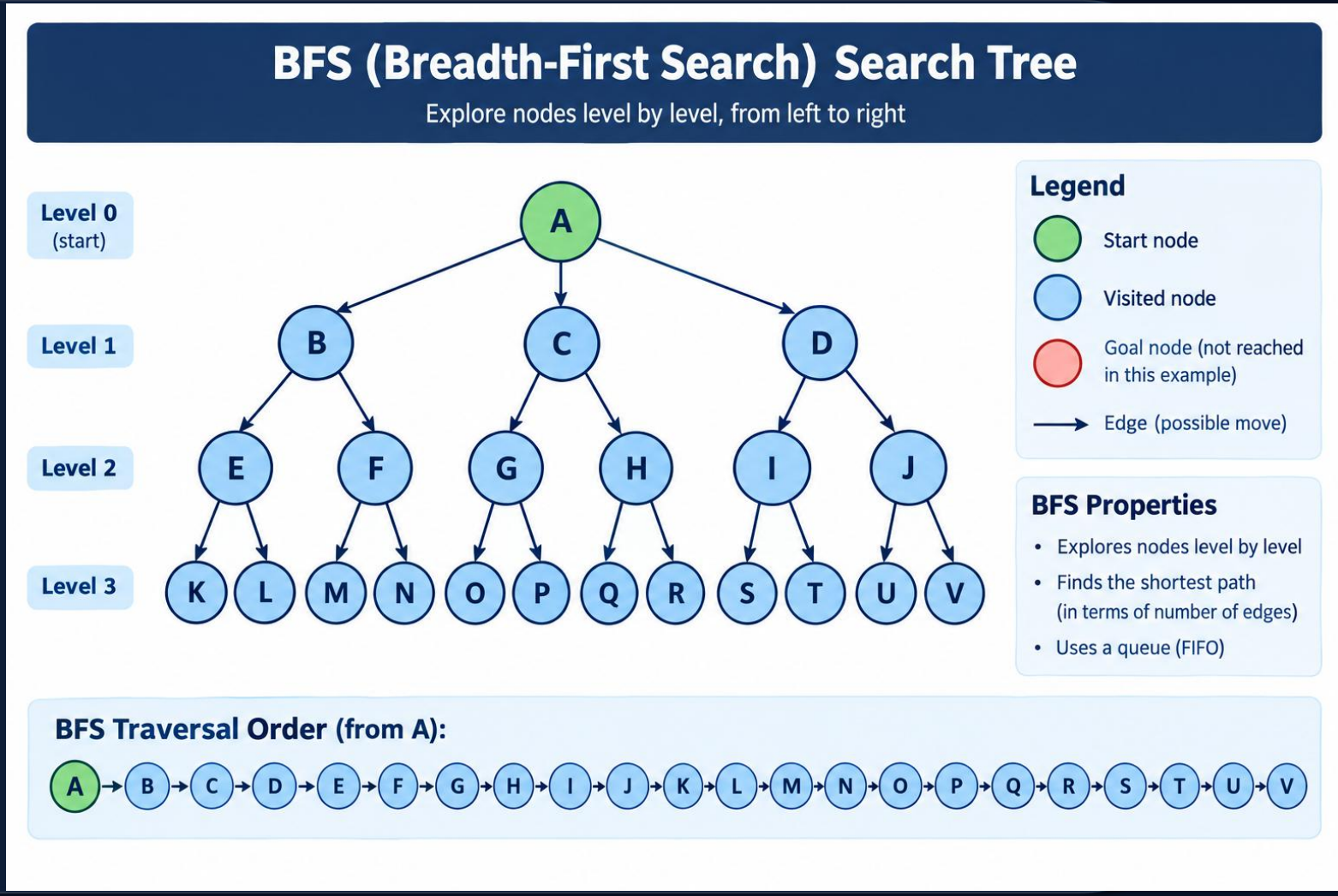
BFS — Breadth-First Search: idea

Uninformed search

BFS explores everything at the current depth before going deeper.

Strategy: expand a shallowest node first

Implementation: Fringe is a FIFO queue



BFS: queue basics

Step 1: understand the data structure

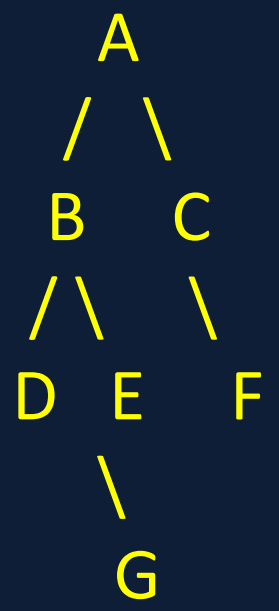
```
graph = {
    "A": ["B", "C"],
    "B": ["D", "E"],
    "C": ["F"],
    "D": [],
    "E": ["G"],
    "F": [],
    "G": []
}

start = "A"
goal = "G"
```

[] → list / indexing

() → tuple or function call

{} → set/dictionary literal



Start: A

Queue: [A]

Expand A → B, C

Queue: [B, C]

Expand B → D, E

Queue: [C, D, E]

Expand C → F

Queue: [D, E, F]

Expand F → GOAL

Solution: A → C → F

BFS: queue basics

Step 1: understand the data structure

```
from collections import deque

queue = deque()
queue.append(('A', ['A']))

state, path = queue.popleft()

('A', ['A']) → state, path
append(x) → add to the right
popleft() → remove from the left
state = "A", path = ["A"]) → Queue:empty
```

BFS: complete code

Runnable implementation

```
from collections import deque

def bfs(graph, start, goal):
    queue = deque([(start, [start])])

    while queue:
        state, path = queue.popleft()

        print("Exploring:", state)

        if state == goal:
            return path

        for next_state in graph[state]:
            queue.append(
                (next_state, path + [next_state])
            )
    return None
```

```
result = bfs(graph, "A", "G")
print("Solution:", result)
```

`queue = deque([(start,[start])])` → create frontier with one plan

`while queue:` → continue while queue is non-empty

`state, path = queue.popleft()` → unpack the oldest plan

`if state == goal:` → equality test

`return path` → stop and give the solution back

`path + [nxt]` → create a new list; do not modify the old path

`not in` → membership test



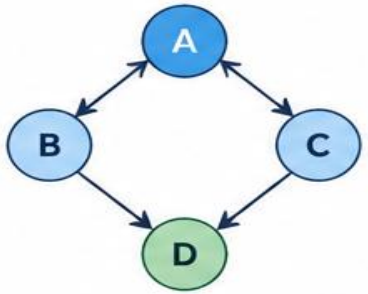
BFS: The Role of the Visited Set

The visited set remembers which states **have already been** explored, preventing repeated visits, avoiding cycles, **and making the search more efficient.**

Without Visited Set

Same state can be added to the queue multiple times.

Graph:



Start: A
Goal: D

Edges:
A → B, A → C
B → D, C → D

BFS Execution (without visited):

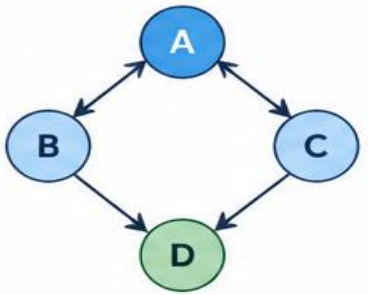
1	**Start** Queue: [A]	Explore A → add B and C	**Queue:** B C (A → B, A → C)
2	**Explore B** Queue: [C, D]	B → add D	**Queue:** C D D (A → C, A → B → D, C → D)
3	**Explore C** Queue: [D, D]	C → add D again (duplicate!)	**Queue:** D D (A → B → D, C → D)
4	**Explore D** Queue: [D]	D is the goal → return path	**Queue:** D (A → B → D)

Problem: Without a visited set, the same state (D) is added twice.
In graphs with cycles, this can lead to infinite loops or unnecessary work.

With Visited Set

Each state is added only once. No duplicates, no cycles.

Graph:



Start: A
Goal: D

Edges:
A → B, A → C
B → D, C → D

BFS Execution (with visited set):

1	**Start** Queue: [A] Visited: {A}	Explore A → add B and C	**Queue:** B C (A → B, A → C) Visited: {A, B, C}
2	**Explore B** Queue: [C, D] Visited: {A, B, C, D}	B → add D (D not visited)	**Queue:** C D (A → C, A → B → D) Visited: {A, B, C, D}
3	**Explore C** Queue: [D] Visited: {A, B, C, D}	C → D is already visited (skip it)	**Queue:** D D (A → B → D) Visited: {A, B, C, D}
4	**Explore D** Queue: [] Visited: {A, B, C, D}	D is the goal → return path	**Queue:** [] (A → B → D) Visited: {A, B, C, D}

Result: Each state is explored only once.
The visited set prevents duplicates and handles cycles safely.

BFS: strengths and limits

Practical choice

Strengths

BFS finds the shortest path in terms of number of actions. It does not find the least-cost path. We will now cover a similar algorithm which does find the least-cost path.

- Complete under standard finite assumptions
- Optimal for equal-cost edges
- Simple

Limits

- Large frontier
- Ignores different costs
- May explore many irrelevant states

Use BFS when “fewest actions” is the objective.

DFS — Depth-First Search: idea

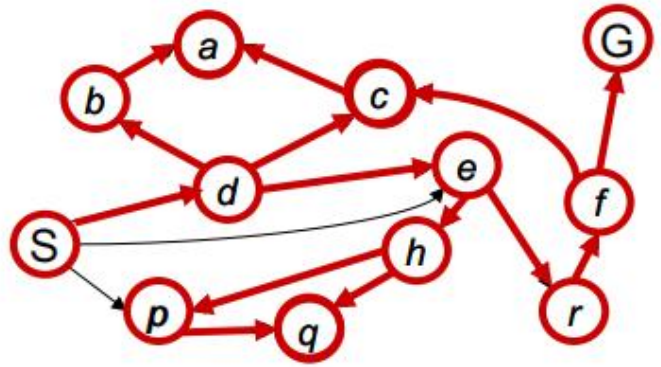
Go as deep as possible before coming back

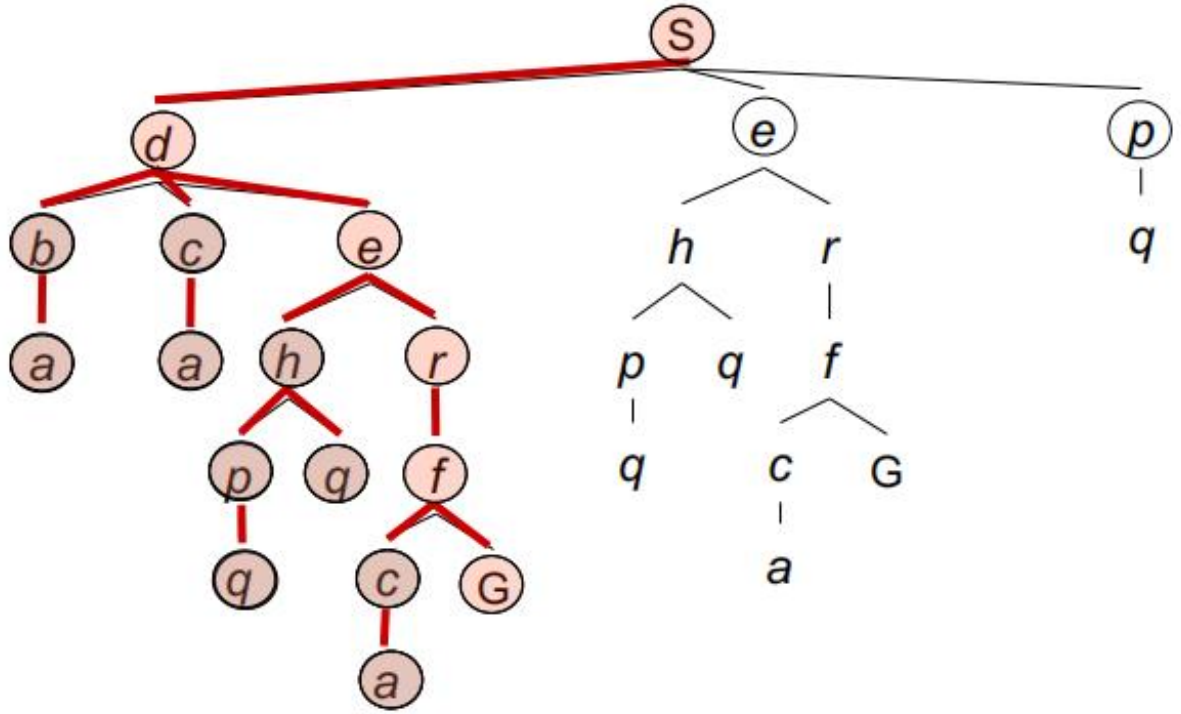
Depth-First Search expands a deepest frontier node first.

Start at a node → go as deep as possible → reach a dead end → back → try another branch

Use a LIFO stack: Last In, First O

It can find a solution withc exploring every shallow alternativ but it is not generally optimal.





```
def dfs(graph, start, goal):
    stack = [(start, [start])]
    visited = set()

    while stack:
        state, path = stack.pop()
        print("Exploring:", state)

        if state == goal:
            return path

        if state not in visited:
            visited.add(state)

            for next_state in graph[state]:

                if next_state not in visited:

                    stack.append(
                        (next_state, path + [next_state])
                    )
    return None
```

stack.pop() → remove last item

visited = {A, B, D} → prevents DFS from repeatedly exploring the same states.

node in visited → Boolean membership

while stack → Keep searching while there is something in the stack

for nxt in ... → iterate through successors

The single biggest strategy change is:
BFS uses popleft(); DFS uses pop().

```
result = bfs(graph, "A", "G")
print("Solution:", result)
```

Same search problem, different fringe policy.

BFS: FIFO → broad layers

DFS: LIFO → deep branch

BFS is attractive for shallow equal-cost solutions.

DFS can save memory in many tree-search settings, but can return a longer solution.

DFS vs BFS

Same Goal. Different Paths.

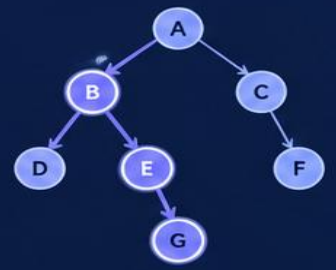
Two search strategies. One problem. Very different approaches.

DFS Go deep first. ←

BFS Go wide first. ←

DFS (Depth-First Search)

Explores one branch as deeply as possible before backtracking to try another branch.



Exploration order (example):

1. A
2. B
3. D
4. E
5. G (goal found!)

G is found after going deep into B's branch.

Step-by-step (stack)

1	Start: [A]	
2	Explore A → push B, C	[C, B]
3	Explore B → push D, E	[C, E, D]
4	Explore D (dead end)	[C, E]
5	Explore E → push G	[C, G]
6	Explore G (goal!)	[]

Stack (LIFO)



Python implementation

```
def dfs(graph, start, goal):
    stack = [(start, [start])]
    visited = set()
    while stack:
        state, path = stack.pop()
        if state == goal:
            return path
        if state not in visited:
            visited.add(state)
            for next_state in graph[state]:
                if next_state not in visited:
                    stack.append((next_state, path + [next_state]))
    return None
```

Quick Comparison

	DFS	BFS
Search order	Deep first	Wide first
Data structure	Stack (LIFO)	Queue (FIFO)
Path guarantee	Not guaranteed (shortest path)	Guaranteed (shortest path in unweighted graph)
Memory usage	Lower	Higher
Time complexity	$O(b^{\wedge m})$	$O(b^d)$
Best for	Large/deep search spaces, when memory is limited	Finding shortest paths, when memory is sufficient

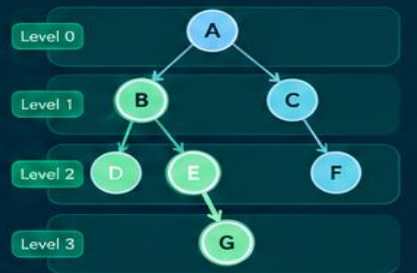
= visited (DFS) = visited (BFS)

= exploration path (DFS)

= exploration path (BFS)

BFS (Breadth-First Search)

Explores all neighbors at the current depth before moving to the next level.



Exploration order (example):

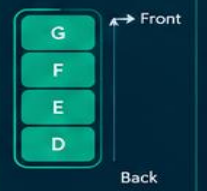
1. A
2. B, C
3. D, E, F
4. G (goal found!)

G is found when reaching level 3.

Step-by-step (queue)

1	Start: [A]	[B, C]
2	Explore A → push B, C	[B, C]
3	Explore B → push D, E	[C, D, E]
4	Explore C → push F	[D, E, F]
5	Explore D (no children)	[E, F]
6	Explore E → push G	[F, G]
7	Explore F (no children)	[G]
8	Explore G (goal!)	[]

Queue (FIFO)



Python implementation

```
from collections import deque

def bfs(graph, start, goal):
    queue = deque([start, [start]])
    visited = set([start])
    while queue:
        state, path = queue.popleft()
        if state == goal:
            return path
        for next_state in graph[state]:
            if next_state not in visited:
                visited.add(next_state)
                queue.append((next_state, path + [next_state]))
    return None
```

Key Takeaways

DFS
- Uses a stack
- Goes deep first
- Low memory, no shortest path guarantee

BFS
- Uses a queue
- Goes wide first
- Finds shortest path (unweighted), higher memory usage

Both are simple, powerful, and widely used in AI, robotics, and game playing!

Different strategies. Same destination.

DFS vs BFS

Same Goal. Different Paths.

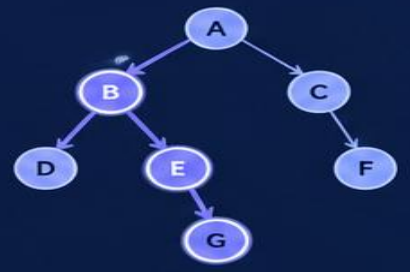
Two search strategies. One problem. Very different approaches.

DFS Go deep first. ←

BFS Go wide first. ←

DFS (Depth-First Search)

Explores one branch as deeply as possible before backtracking to try another branch.



Exploration order (example):

1. A
2. B
3. D
4. E
5. G (goal found!)

G is found after going deep into B's branch.

Step-by-step (stack)

1	Start: [A]	
2	Explore A → push B, C	[C, B]
3	Explore B → push D, E	[C, E, D]
4	Explore D (dead end)	[C, E]
5	Explore E → push G	[C, G]
6	Explore G (goal!)	[]

Stack (LIFO)



Python implementation

```
def dfs(graph, start, goal):
    stack = [(start, [start])]
    visited = set()
    while stack:
        state, path = stack.pop()
        if state == goal:
            return path
        if state not in visited:
            visited.add(state)
            for next_state in graph[state]:
                if next_state not in visited:
                    stack.append((next_state, path + [next_state]))
    return None
```

Quick Comparison

	DFS	BFS
Search order	Deep first	Wide first
Data structure	Stack (LIFO)	Queue (FIFO)
Path guarantee	Not guaranteed (shortest path)	Guaranteed (shortest path in unweighted graph)
Memory usage	Lower	Higher
Time complexity	$O(b^m)$	$O(b^d)$
Best for	Large/deep search spaces, when memory is limited	Finding shortest paths, when memory is sufficient

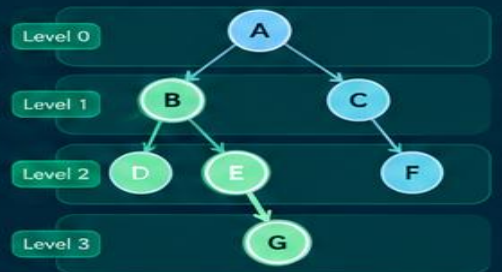
= visited (DFS) = visited (BFS)

= exploration path (DFS)

= exploration path (BFS)

BFS (Breadth-First Search)

Explores all neighbors at the current depth before moving to the next level.



Exploration order (example):

1. A
2. B, C
3. D, E, F
4. G (goal found!)

G is found when reaching level 3.

Step-by-step (queue)

1	Start: [A]	[B, C]
2	Explore A → push B, C	[B, C]
3	Explore B → push D, E	[C, D, E]
4	Explore C → push F	[D, E, F]
5	Explore D (no children)	[E, F]
6	Explore E → push G	[F, G]
7	Explore F (no children)	[G]
8	Explore G (goal!)	[]

Queue (FIFO)



Python implementation

```
from collections import deque

def bfs(graph, start, goal):
    queue = deque([start, [start]])
    visited = set([start])
    while queue:
        state, path = queue.popleft()
        if state == goal:
            return path
        for next_state in graph[state]:
            if next_state not in visited:
                visited.add(next_state)
                queue.append((next_state, path + [next_state]))
    return None
```

Key Takeaways

DFS
- Uses a stack
- Goes deep first
- Low memory, no shortest path guarantee

BFS
- Uses a queue
- Goes wide first
- Finds shortest path (unweighted), higher memory usage

Both are simple, powerful, and widely used in AI, robotics, and game playing!

Different strategies. Same destination.

Iterative Deepening

Run depth-limited DFS with limit 1, then 2, then 3, …

Goal: combine DFS-like memory use with BFS-like preference for shallow solutions.

Repeated work occurs, but most work is usually near the deepest level searched.

Weighted paths: why BFS fails

Cost-sensitive search

A→B cost 2

A→C cost 8

B→F cost 10

C→F cost 1

Both routes use two actions.

BFS sees equal depth.

UCS compares total cost:

A-B-F = 12

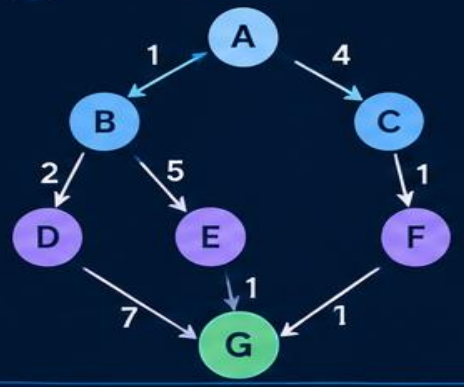
A-C-F = 9

Uniform Cost Search (UCS)

Finds the lowest-cost path using a priority queue (min-heap).

Key idea:
Always expand the node with the lowest total cost.

1 Example Graph

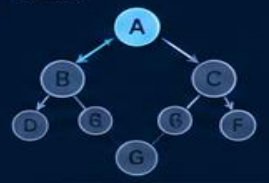


Start: A
Goal: G

Path cost (example):
A → B → E → G
1 + 5 + 1 = 7

2 Step-by-Step Execution

1 Start



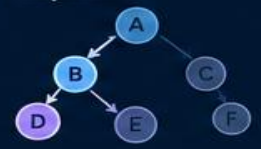
Queue
A 0)]

2 Expand A



Queue
B (B, 1)
C (C, 4)

3 Expand B



Queue
D (D, 3)
E (E, 6)
C (C, 4)

4 Expand D



Queue
E (E, 6)
C (C, 4)
G (G, 10)

5 Expand C



Queue
E (E, 6)
F (F, 5)
G (G, 10)

6 Expand F



Queue
E (E, 6)
G (G, 6)

7 Expand E



Queue
G (G, 6)

8 Expand G



Goal reached!
Path cost: 6
A → B → E → G

3 Python (simplified)

```
def ucs(graph, start, goal):
    pq = [(0, start, [start])]
    visited = set()

    while pq:
        cost, node, path = heapq.heappop(pq)
        if node == goal:
            return path

        if node not in visited:
            viisted.add(node)
             for next_node, edge_cost in graph[node]:
                heapq.heappush
                        pq,  (cost + edge_cost,
                              next_node,
                              path + [next_node])

    return None
```

How it works

- Priority queue (min-heap)
- Expand lowest cost first
- Uses visited set
- Finds optimal path (if costs ≥ 0)

UCS vs. BFS vs. DFS

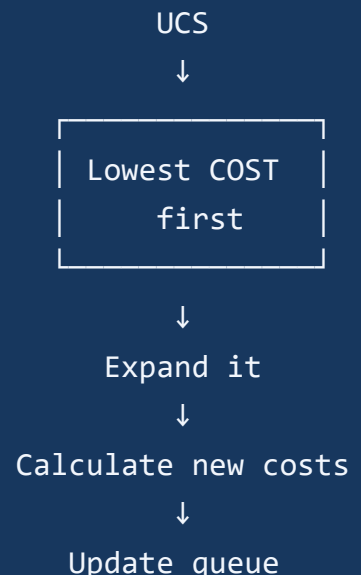
Feature	UCS	BFS	DFS
Order	Lowest cost	Level by level	Deep first
Data structure	Priority queue	Queue	Stack
Optimal path?	Yes (non-negative)	Yes (unweighted)	No

When to use UCS?

- ✓ Different edge costs
- ✓ Find the cheapest path
- ✓ Non-negative costs

Best when costs matter!

```
def ucs(graph, start, goal):
    pq = [(0, start, [start])]
    best = {start: 0}
    while pq:
        cost, node, path = heapq.heappop(pq)
        if node == goal:
            return cost, path
        for nxt, step in graph[node]:
            new_cost = cost + step
            if new_cost < best.get(nxt, float('inf')):
                best[nxt] = new_cost
                heapq.heappush(pq, (new_cost, nxt, path +
[nxt]))
    return None
```




```
pq = []
heapq.heappush(pq, (0, 'A'))
heapq.heappush(pq, (5, 'B'))
cost, node = heapq.heappop(pq)
```

Smallest priority is removed first.

`heapq.heappush(pq, item)` → insert
`heapq.heappop(pq)` → remove smallest

`cost + step` → add path costs

`best.get(nxt, float('inf'))`
→ retrieve known cost; use infinity if absent

`<` → “is less than”; used to detect a better route

UCS: understand g(n)

Cost contour intuition

g(n) = exact cost from START to n.

A→B costs 2
B→C costs 4

Therefore g(C)=6.

UCS always prioritizes the frontier node with the smallest g(n).

Under standard positive-cost assumptions, UCS is complete and optimal.

From UCS to A*

Bridge to informed search

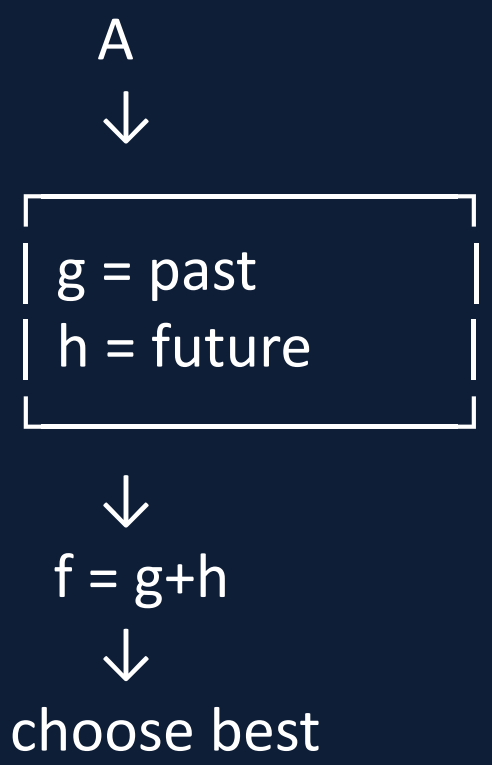
UCS asks: “What have I paid so far?”

A* additionally asks: “What might remain? “How much farther do I probably have to go?””

g(n) = exact cost so far

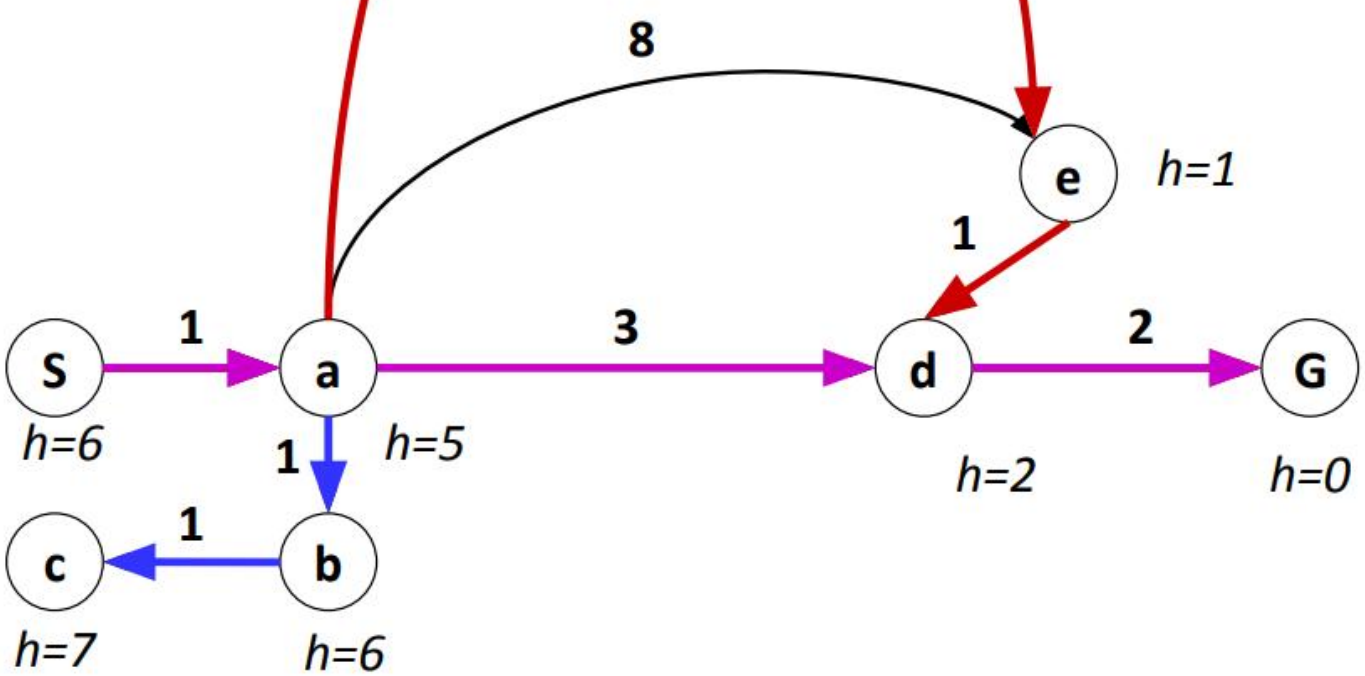
h(n) = estimated remaining cost

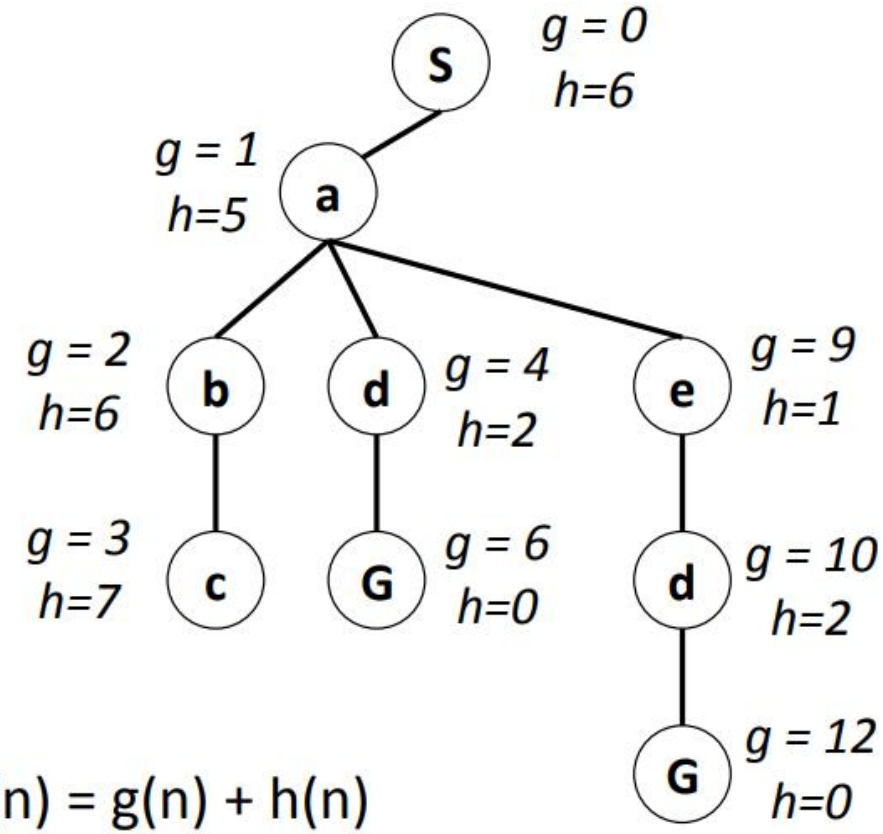
$$f(n) = g(n) + h(n)$$



Combining UCS and Greedy

- Uniform-cost orders by path cost, or *backward cost* g(n)
- Greedy orders by goal proximity, or *forward cost* h(n)





- A* Search orders by the sum, or *total cost*: f(n) = g(n) + h(n)

Example: Teg Grenager

Major matter

For A*, the most important thing is the total path cost, while the heuristic helps A* find that low-cost path efficiently.

The key formula is:

$$f(n)=g(n)+h(n)$$

g(n) = actual cost so far

h(n) = estimated cost to the destination **(**A heuristic is an estimate that helps a search algorithm decide which direction looks more promising**)**

f(n) **=** estimated total cost

So is A* looking for shortest or cheapest? It depends on what "shortest" means.

Major matter

If every step has the same cost:

A → B → C → G

4 steps

Then shortest number of steps = cheapest path.

So A* finds the shortest path.

HOWEVER

If edges have different costs:



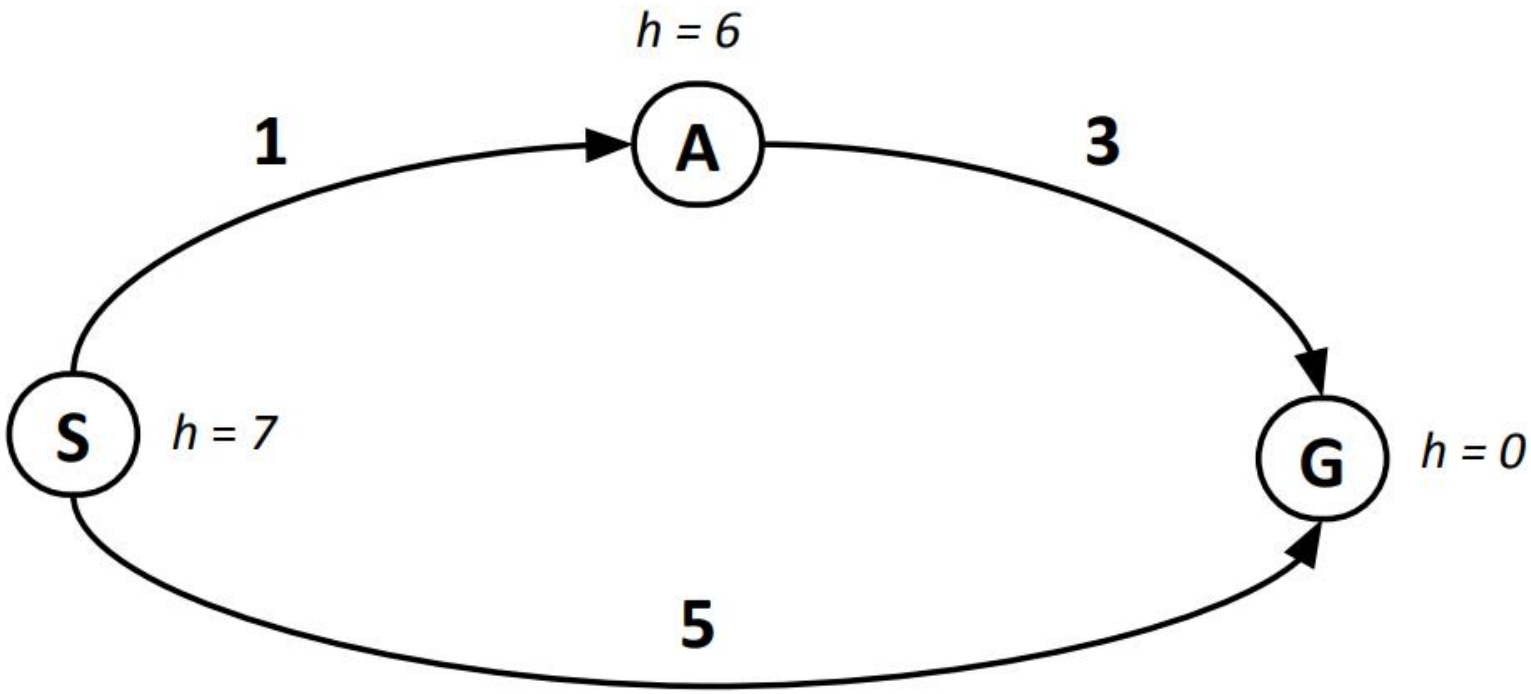
A* cares about:

A → B → G = 2

A → G = 2

It chooses based on total cost, not simply number of edges.

Is A* Optimal?

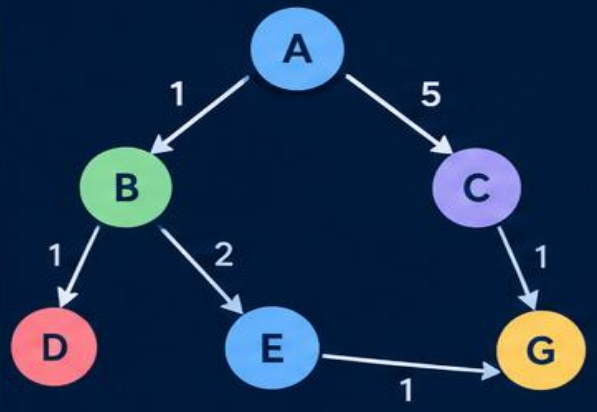


- What went wrong?
- Actual bad goal cost < estimated good goal cost
- We need estimates to be less than actual costs!

Admissible Heuristics

A heuristic h is admissible (optimistic) if: $0 <= h(n) <= h^*(n)$ ($h^*(n)$ - is the true cost to a nearest goal)

Example Graph



True cost to goal (h*)

Node	h* (actual remaining cost)
A	3
B	2
C	1
D	3
E	1
G	0

These are the real cheapest costs to reach G.

Heuristic Values

Node	Admissible (good)	Inadmissible (pessimistic)
A	3	5
B	2	4
C	1	3
D	3	6
E	1	2
G	0	0

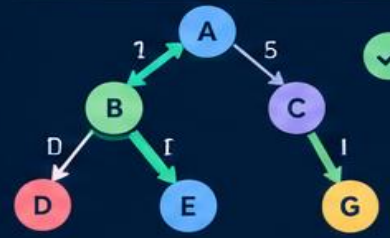
Admissible
$h(n) \le h^*(n)$
(never overestimates)

Inadmissible
$h(n) > h^*(n)$
(overestimates)

A* with Admissible Heuristic

Finds the optimal path

Path	Total cost
A → B → E → G	3



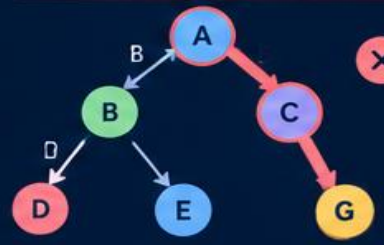
Optimal cost = 3

A* expands nodes based on $f(n) = g(n) + h(n)$ and finds the cheapest path.

A* with Inadmissible Heuristic

May find a suboptimal path

Path	Total cost
A → C → G	6



Suboptimal cost = 6

The heuristic overestimates the cost, so A* is misled and chooses the wrong path.

What happens with an inadmissible heuristic?

1. A* may choose a path that looks cheaper (according to h), but is actually more expensive.
2. It can miss the optimal solution.
3. The search is still complete (if step costs are non-negative), but no longer optimal.

Key Takeaways

- **Inadmissible heuristics** (overestimate) can lead to **suboptimal solutions.**
- They are still **complete** (with non-negative costs).
- **Admissible heuristics guarantee optimality.**
- The heuristic should be optimistic, not **pessimistic.**

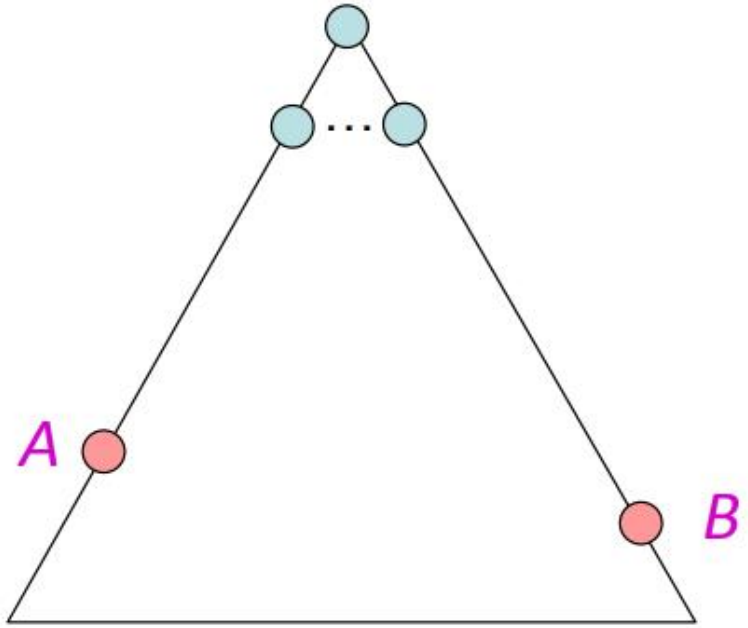
Optimality of A* Tree Search

Assume:

- A is an optimal goal node
- B is a suboptimal goal node
- h is admissible

Claim:

- A will be chosen for expansion before B



Optimality of A* Tree Search

Proof:

- Imagine B is on the frontier
- Some ancestor n of A is on the frontier,

too (maybe A itself!)

Claim:

n will be expanded before B

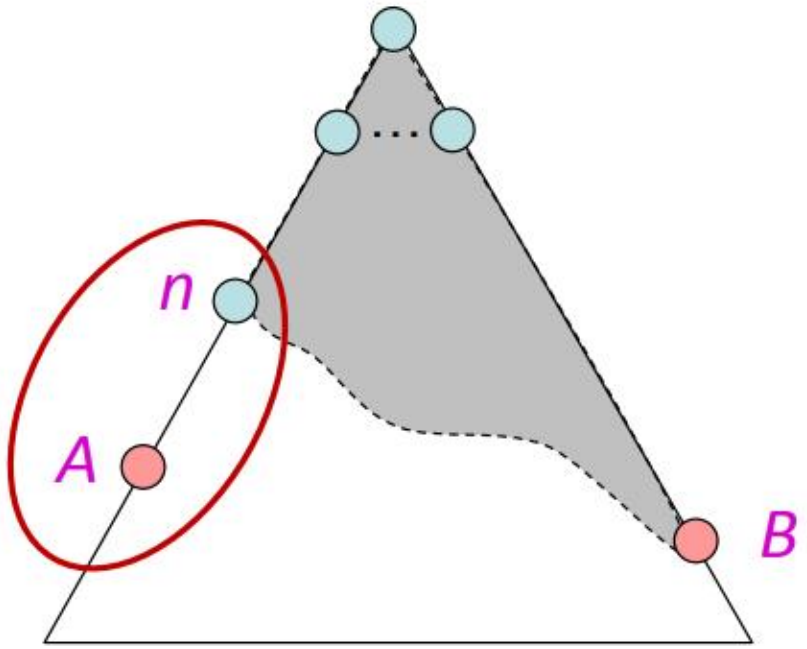
1. $f(n) \leq f(A)$

here

$f(n) = g(n) + h(n)$ Definition of f-cost

$f(n) \leq g(A)$ Admissibility of h

$g(A) = f(A)$ $h = 0$ at a goal



Optimality of A* Tree Search, Blocking

Proof:

- Imagine B is on the frontier
- Some ancestor n of A is on the frontier,

too (maybe A itself!)

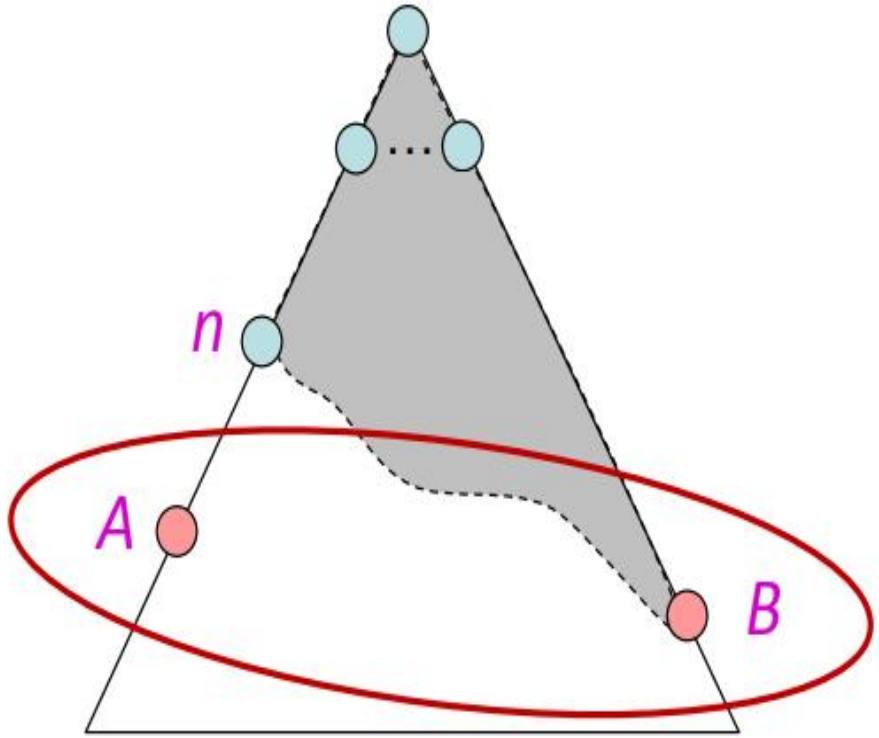
- Claim: n will be expanded before B

1. f(n) ≤ f(A)
2. f(A) < f(B)

here

g(A) < g(B) Suboptimality of B

f(A) < f(B) h = 0 at a goal



Optimality of A* Tree Search, Blocking

Proof:

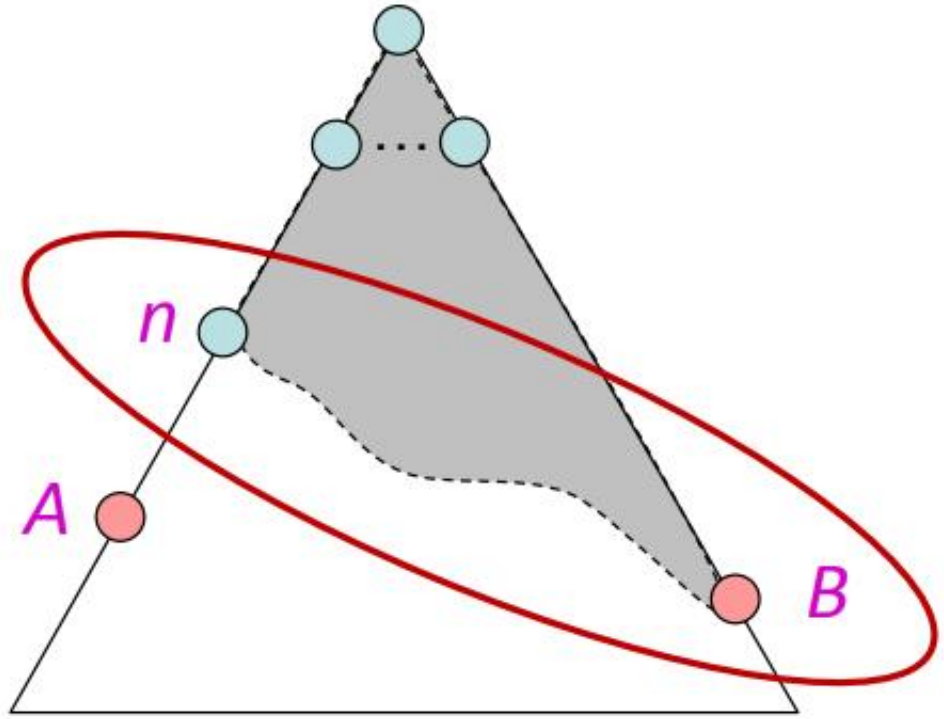
- Imagine B is on the frontier
- Some ancestor n of A is on the frontier, too (maybe A itself!)
- Claim: n will be expanded before B

1. f(n) ≤ f(A)
2. f(A) < f(B)
3. n is expanded before B

- All ancestors of A are expanded before B
- A is expanded before B
- A* tree search is optimal

here

f(n) ≤ f(A) < f(B)



Simple numerical example

Simple numerical example

Suppose we have three candidate nodes:

Node	g(n)	h(n)	f(n)
A	2	10	12
B	5	4	9
C	7	3	10

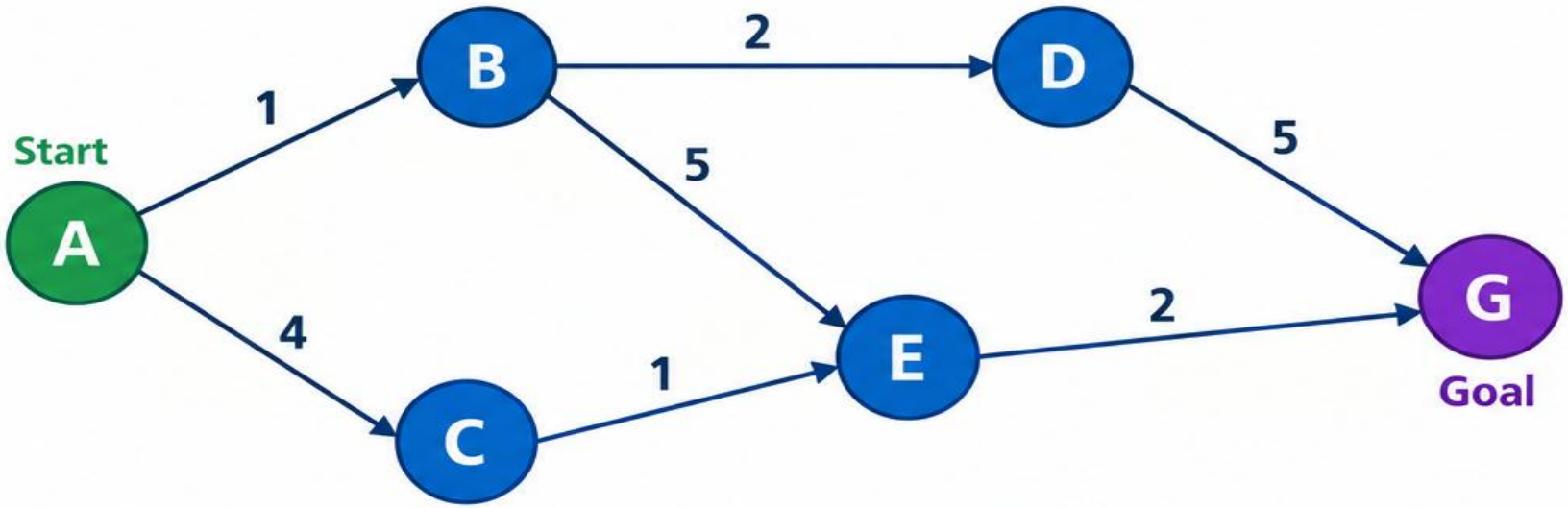
A* chooses B, because:

f(B)=5+4=9

Even though B has already cost 5, it looks more promising overall.

A* Python Example – Graph

Find the shortest path from A (start) to G (goal) using A* search.



Heuristic Values h(n)

Node	h(n) (estimated cost to goal)
A	6
B	5
C	3
D	4
E	2
G	0

Python representation of the graph:

```
graph = {
    "A": [("B", 1), ("C", 4)],
    "B": [("D", 2), ("E", 5)],
    "C": [("E", 1)],
    "D": [("G", 5)],
    "E": [("G", 2)],
    "G": []
}
```

Heuristic dictionary:

```
heuristic = {
    "A": 6,
    "B": 5,
    "C": 3,
    "D": 4,
    "E": 2,
    "G": 0
}
```

Legend

- Start node
- Goal node
- Intermediate node
- → Edge (with cost)
- h(n) Heuristic value (in table above)

A*: decision example

Calculate f(n) before choosing

```
Node A: g=4, h=8 → f=12
Node B: g=7, h=2 → f=9

A* expands B because 9 < 12.

It is willing to pay more now if the estimated total route looks better.
```

A* Search – Grid Pathfinding Example

Find a path from the start to the goal on a grid with obstacles.

A* uses $f(n) = g(n) + h(n)$

where:

- $g(n)$ – cost from start to n
- $h(n)$ – estimated cost to goal
- $f(n)$ – total estimated cost

The Grid

```
grid = [
    '.....',
    '.###.',
    '...#.',
    '.#...',
    '.....'
]

start = (0, 0)
goal = (4, 4)
```



S = Start (0, 0)

G = Goal (4, 4)

= Free space (.)

= Blocked (#)

= Path (result)

Python Implementation

```python
import heapq

def manhattan(a, b):
    return abs(a[0] - b[0]) + abs(a[1] - b[1])

def get_neighbors(pos, grid):
    rows, cols = len(grid), len(grid[0])
    r, c = pos
    directions = [(-1, 0), (1, 0), (0, -1), (0, 1)]
    for dr, dc in directions:
        nr, nc = r + dr, c + dc
        if 0 <= nr < rows and 0 <= nc < cols:
            if grid[nr][nc] != '#':  # not blocked
                yield (nr, nc)

def a_star(grid, start, goal):
    frontier = []  # priority queue (f, g, position)
    heapq.heappush(frontier, (manhattan(start, goal), 0, start))

    came_from = {start: None}
    cost_so_far = {start: 0}

    rows, cols = len(grid), len(grid[0])

    while frontier:
        f, g, current = heapq.heappop(frontier)
        if current == goal:
            break

        for neighbor in get_neighbors(current, grid):
            new_cost = g + 1  # each move costs 1
            if neighbor not in cost_so_far or new_cost < cost_so_far[neighbor]:
                cost_so_far[neighbor] = new_cost
                f = new_cost + manhattan(neighbor, goal)
                heapq.heappush(frontier, (f, new_cost, neighbor))
                came_from[neighbor] = current

    return came_from, cost_so_far[goal]
```

Result: Found Path



Path (coordinates):

[(0, 0), (0, 1), (0, 2), (0, 3), (0, 4), (1, 4), (2, 4), (3, 4), (4, 4)]

Total cost: 8

Step-by-Step Visualization





Neighbors: operators

Boolean logic controls legality

yield → produce one result at a time

new_row = row + dr, new_col = col + dc → program generates possible actions

len(grid) → number of rows

len(grid[0]) → number of columns

`heapq.heappush → insert`

`heapq.heappop → remove smallest`

```
directions = [
    (-1, 0),  # up
    (1, 0),   # down
    (0, -1),  # left
    (0, 1)    # right
]
```

The condition prevents moving outside the grid or into a wall.

A*: step-by-step

What each iteration accomplishes

1. Pop smallest f
2. Check goal
3. Generate neighbors
4. Compute tentative g
5. If new g is better, update
6. Compute f = g+h
7. Push frontier item

The dictionary came_from remembers how each state was reached.

Reconstructing the Path

How A* turns parent links into the final route

The search algorithm tells us which node came from which parent. We just need to follow the links backward from the goal to the start.

1. How it works

1. During the search, we store a dictionary `came_from`, where each node points to its parent node.
2. After reaching the goal, we start from the goal and follow the parent links backward.
3. We collect the nodes in a list, then reverse it to get the correct path from start to goal.

Key idea

The parent links form a tree from the start to every explored node. The goal node is just one of the leaves.

Python Implementation

```python
def reconstruct_path(came_from, start, goal):
    """Reonstructs the path from start to goal."""
    current = goal
    path = [current]

    while current != start:
        current = came_from[current]
        path.append(current)

    path.reverse()
    return path
```

Example: Reconstructing the Path

came_from (partial view)

```
came_from = {
 (0, 0): None,     # start
 (0, 1): (0, 0),
 (0, 2): (0, 1),
 (1, 2): (0, 2),
 (2, 2): (1, 2),   # goal
 ...
}
```

Result

Path (start → goal):

```
[(0, 0),
 (0, 1),
 (0, 2),
 (1, 2),
 (2, 2)]
```

(reversed at the end)

Step-by-Step Path Reconstruction (Grid Example)

1. Goal node
Start from the goal (2, 2).

2. Follow parent
Go to the parent of (2, 2): (1, 2).

3. Continue
Next parent: (0, 2).

4. Continue again
Next parent: (0, 1).

5. Final path
Reverse the list to get the path from start to goal.



Final Path (coordinates)

[(0, 0), (0, 1), (0, 2), (1, 2), (2, 2)]

S Start
G Goal
Blocked cell
Path

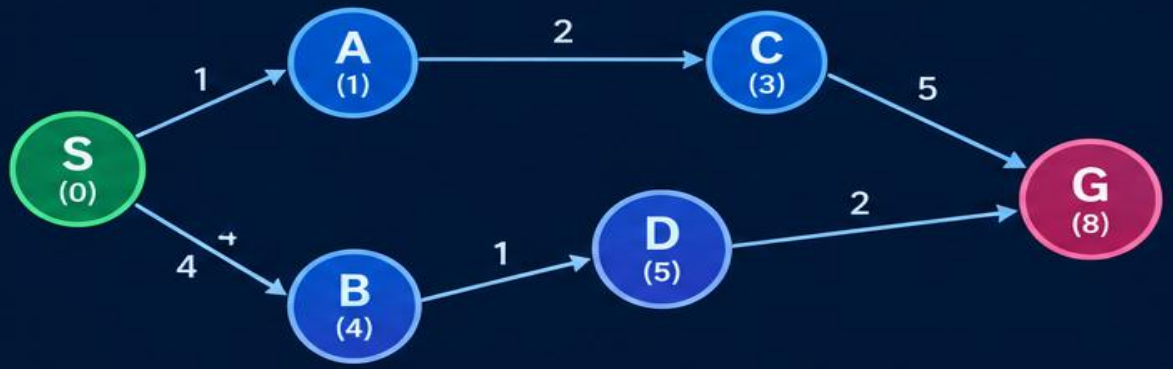
By following parent links and reversing the path, we recover the optimal route found by A*.

A* vs UCS

Same goal. Different strategy.

Uniform Cost Search (UCS)

Expands the node with the lowest path cost (g(n)).



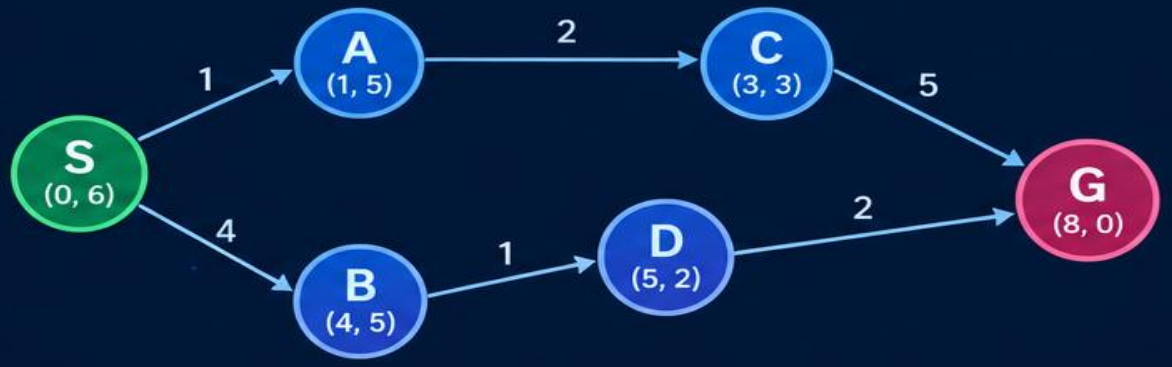
Exploration order (UCS):

S → A → B → D → C → G

Path found:
S → B → D → G
Cost: 8

A* Search

Expands the node with the lowest f(n) = g(n) + h(n).



Exploration order (A*):

S → A → C → D → G

Path found:
S → A → C → G
Cost: 8

	UCS	A*
Selection criterion	Lowest g(n)	Lowest f(n) = g(n) + h(n)
Uses heuristic	✗	✓
Optimal (with admissible h)	✓	✓
Memory usage	Higher	Lower (often)

A* is like UCS with a compass.

A* vs UCS

Key relationship

If $h(n)=0$:

$f(n)=g(n)+0=g(n)$

So A* behaves like UCS.

A useful heuristic can reduce exploration because it incorporates information about the goal.

An overestimating heuristic may be faster but can lose optimality guarantees.